The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...
Module 2: WHY THIS WO...
Module 3: FOLDER ARCHI...
3.1 The Full Walkthrough (23 ...
3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...
Module 4: CLAUDE CODE ...
SECTION 5: WHERE TO G...

1.3 How to Structure Any Prompt

What You'll Get From This

A framework for building prompts that get useful results on the first try. Five parts. Works for writing, coding, research, or anything else you throw at Claude.

The Lesson

A prompt is an instruction set. The clearer the instruction, the better the result. Most people write prompts the way they send a text message. That works for simple questions. It falls apart the moment you want Claude to do real work.

If you went through **Section 1.2: Your First Folder**, you already know that giving Claude structured context changes the output. The folder handles the big picture: who you are, what the project is, what good looks like. This lesson handles the individual ask. Each time you need Claude to do something specific, how do you phrase it so you get something useful back?

Five parts. You will not use all five every time. But knowing them means you always know which one is missing when the output is not right.

2

depth, and the assumptions it makes.

> You are a senior copywriter who writes for B2B SaaS companies.

> You are a research assistant helping me prepare a literature review.

> You are a Python developer who writes clean, well-documented code.

If you set up a CLAUDE.md file in 1.2, this part is already handled for your project. The file does this work for you in every conversation. But for one-off tasks or tasks where you need Claude to shift roles, you add identity at the top of your prompt.

One note: identity is the part most people skip. They jump straight to "write me a blog post" without telling Claude who is writing it. The output is always more generic without it.

Part 2: Task — What needs to get done?

Be specific. "Help me write something" is vague. "Write a 200-word product description for [X] targeting [Y] audience" gives Claude something to work with.

Good tasks have three things:

- **A clear action — write, review, analyze, compare, build, fix, summarize**
- **A defined scope — how long, how many, what format, what section**
- **Enough detail that someone unfamiliar with your project could attempt it**

If you read your task out loud and a stranger could not start working on it without asking you five follow-up questions, the task is too vague.

know?

This is the background. The constraints. The audience. Prior decisions. Relevant data. Anything Claude needs to do the task well that it would not know on its own.

If you are using the folder structure from 1.2, your CONTEXT.md handles the project-level stuff. But for individual prompts, you can also give context directly:

> We are a 15-person startup. Our customers are mid-market HR directors. We just launched a feature that automates onboarding checklists. Here is the feature spec: [paste spec]

The more relevant context you give, the less Claude has to guess. Guessing is where AI outputs go sideways. If the output feels generic or off-target, the fix is almost always more context, not a better prompt.

Part 4: Constraints — What should Claude avoid?

This is the part where most people leave value on the table. Telling Claude what you do NOT want is just as useful as telling it what you do want.

> Do not use jargon. Write at an 8th grade reading level.

> Do not suggest solutions that require a paid API. Everything should use free tools.

> Keep it under 300 words. No bullet points. Write in paragraphs.

> Do not start with "In today's world" or any variation of it.

you did not set.

Part 5: Output Format — What should the result look like?

Tell Claude the shape of the answer. A list? A table? Three options to choose from? A code block with comments? A draft with placeholder sections?

> Give me three headline options, each under 10 words, followed by a one-sentence explanation of the angle.

> Return this as a markdown table with columns for Task, Owner, Deadline, and Status.

> Write the first draft with [PLACEHOLDER] wherever I need to fill in company-specific details.

> Give me the answer as a numbered list, then add a one-paragraph summary at the end.

Output format is the difference between getting something you can use immediately and getting something you have to reformat for 20 minutes.

When to use which parts

You do not need all five in every prompt. Here is the shortcut:

If your task is... You probably need... Simple and quick (rename this, fix this typo) Task only Creative (write this, design this) Identity + Task + Constraints + Output Format Complex (build this system, analyze this data) All five Ongoing (a project over many messages) Identity and Context in your folder files. Task and Constraints in each prompt.

Notice the last row. That is where the folder structure from 1.2 and this prompting framework connect. Your files carry the

time, what shape the output should take).

The folder is memory. The prompt is direction. They work together.

Chunking: breaking big projects into prompts

If your project is bigger than a single prompt can handle, break it into steps.

The rule: each prompt should ask for one clear thing.

Too much at once: "Write me a full marketing strategy with a content calendar, email sequences, and social posts for the next quarter."

Claude will try to do all of it. The output will be shallow across the board because it is spreading its effort too thin.

Chunked into steps:

1. "Here is our product and audience. Outline the three main themes for Q2 content."
2. [You review. You adjust. You pick a direction.]
3. "Take theme 1 and draft a 4-week content calendar."
4. [You review. You adjust.]
5. "Write the first email in the nurture sequence for theme 1."

Each step builds on the last. You review and correct between steps. Claude gets cleaner input each time because you narrowed the scope. And if something goes wrong at step 3, you only redo step 3, not the whole thing.

This is the same principle behind the folder architecture. You do not dump everything into one giant file. You break it into layers. Prompting works the same way.

Chunking data: feeding Claude large inputs

Step 1: Give Claude the structure first.

> "I am going to give you a 40-page report in sections. Here is the table of contents. I will send each section one at a

> time."

Step 2: Feed sections in order. After each one, you can ask Claude to confirm what it picked up.

> "Here is Section 3. What are the key claims in this section?"

Step 3: After the last section, ask Claude to work across everything.

> "Now that you have all sections, summarize the three strongest arguments and the two weakest."

This works better than pasting 50 pages into one message. Claude handles structured, sequential input more reliably than a wall of text with no markers.

One more trick: if your data is in a spreadsheet or table format, keep it in that format. Do not convert it to paragraphs before giving it to Claude. Tables and structured data are already token-efficient. Let Claude read them as-is.

A real example: one prompt, five parts

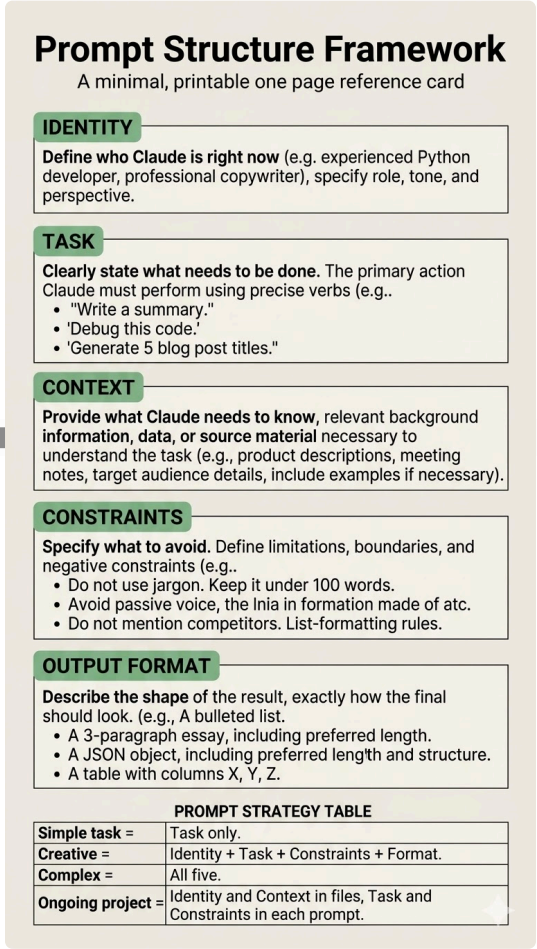
Here is what it looks like when you put all five parts together. This is a single prompt:

```
You are a technical writer who explains complex topics to a
(Identity)

Write a 300-word explanation of how API keys work and why s
(Task)
```

```
they are setting up Claude Code.
(Context)

Do not use jargon without explaining it. Do not assume they
Keep sentences short.
(Constraints)
```

```
Start with a one-sentence analogy, then explain the concept,
three steps to get their first API key. End with a one-line
is easier than it sounds.
(Output Format)
```

You will not always write prompts this long. But when you are getting vague or unhelpful responses, you can look at this structure and ask: which part am I missing?



Prompt Structure Framework

A minimal, printable one page reference card

IDENTITY

Define who Claude is right now (e.g. experienced Python developer, professional copywriter), specify role, tone, and perspective.

TASK

Clearly state what needs to be done. The primary action Claude must perform using precise verbs (e.g..
- "Write a summary."
- 'Debug this code.'
- 'Generate 5 blog post titles."

CONTEXT

Provide what Claude needs to know, relevant background **information, data, or source material** necessary to understand the task (e.g., product descriptions, meeting notes, target audience details, include examples if necessary).

CONSTRAINTS

Specify what to avoid. Define limitations, boundaries, and negative constraints (e.g..
- Do not use jargon. Keep it under 100 words.
- Avoid passive voice, the lnia in formation made of atc.
- Do not mention competitors. List-formatting rules.

OUTPUT FORMAT

Describe the shape of the result, exactly how the final should look. (e.g., A bulleted list.
- A 3-paragraph essay, including preferred length.
- A JSON object, including preferred leng'th and structure.
- A table with columns X, Y, Z.

PROMPT STRATEGY TABLE

Simple task =	Task only.
Creative =	Identity + Task + Constraints + Format.
Complex =	All five.
Ongoing project =	Identity and Context in files, Task and Constraints in each prompt.

Go Deeper

structure from **1.2: Your First Folder** is how you give Claude memory across conversations. Identity and Context live in your files. Task and Constraints live in your prompts.

- Want to understand why structured input changes these output at a fundamental level? **2.3: How a 1953 Word Game**

Explains AI Memory traces prompting back to fill-in-the-blank word games and context windows.

- The **Learning the Abstraction** series (Section 2) goes deeper into how AI processes information and why specific structure matters at every level. Start with the overview at **2.1: Video Text Guide**.
- In Premium and Level 2+ courses, we apply this framework to specific use cases: building automations, writing production content, working with code. Everything builds on what you learned here.

Next Up: Section 2 — Why This Works (The Abstraction Series)

You have tools (**1.1**), a folder (**1.2**), and a framework (1.3). Section 2 explains the ideas underneath all of it. Six videos that trace one throughline: AI is the next layer in a stack that has been building for seventy years. Once you see the pattern, you stop falling for hype and start making better decisions about where AI actually fits.

Matthew Creamer

7 24d • Lesson Check-Ins

Foundations 1.3 Check-In

You learned the framework. Now try it on something you're actually working on. Vote below, then drop your prompt

Poll 387 members have voted

141 79

New comment 3h ago